

Projeto: Contexto Semantico Pre-Filtrado

Reducao de tokens e ganho de velocidade via embeddings no pipeline do Hermes

1. Objetivo

Hoje, quando voce faz uma pergunta ao Hermes, o Kimi recebe o contexto completo do vault ou nenhum contexto. O objetivo deste projeto e inserir uma etapa de pre-filtragem semantica antes de cada chamada ao Kimi: em vez de injetar tudo, injetar apenas as notas mais relevantes para aquela pergunta especifica.

Resultado esperado: menos tokens por chamada, respostas mais rapidas e mais precisas, sem mudar o comportamento do Kimi nem adicionar novos modelos ao pipeline.

2. Fluxo Atual vs Fluxo Proposto

Fluxo atual:

```
Pergunta do usuario → Kimi → busca ampla no vault → resposta
```

Fluxo proposto:

```
Pergunta do usuario → embedding da query (nv-embedcode-7b-v1) → top 5 notas similares → Kimi recebe contexto filtrado → resposta
```

O modelo de embeddings ja esta instalado e operacional desde o projeto anterior. Nenhuma nova dependencia e necessaria.

3. O que precisa ser feito

3.1 Diagnostico inicial

Antes de qualquer codigo, mapear como o run_agent.py injeta contexto hoje. Identificar: onde o system prompt e montado, se ja existe algum mecanismo de busca no vault antes das chamadas ao Kimi, e qual e o tamanho medio do contexto atual em tokens.

Comandos para o diagnostico:

```
grep -n 'system_prompt\|context\|vault\|memory\|obsidian'  
~/.hermes/hermes-agent/run_agent.py | head -40
```

```
grep -n 'def ' ~/.hermes/hermes-agent/run_agent.py | head -30
```

3.2 Criar funcao get_semantic_context()

Criar uma funcao que recebe a query do usuario, gera o embedding via nv-embedcode-7b-v1 com input_type='query', consulta a vec_obsidian e retorna o conteudo das top N notas mais

similares formatado como texto para injeção no contexto.

Parâmetros recomendados: top_n=5, score_threshold=0.3 (ignorar notas muito distantes), max_chars_per_note=1500 (evitar notas gigantes dominarem o contexto).

3.3 Ponto de injeção no run_agent.py

Após o diagnóstico, identificar o ponto exato onde o contexto é montado antes da chamada ao Kimi e inserir a chamada a get_semantic_context(). O contexto filtrado deve ser adicionado como seção específica no system prompt, separada do restante, com cabeçalho claro:

```
## Notas relevantes do vault\n{conteudo_filtrado}
```

3.4 Fallback

Se a API NVIDIA estiver indisponível ou a vec_obsidian estiver vazia, o pipeline continua normalmente sem contexto pré-filtrado. Nunca bloquear a execução do agente por falha no pré-filtro.

3.5 Controle on/off

Adicionar variável de ambiente HERMES_SEMANTIC_CONTEXT=true/false no .env para ligar e desligar o pré-filtro sem modificar código. Útil para comparar comportamento com e sem o filtro ativo.

4. Medição de resultado

Metрика	Como medir
Redução de tokens	Comparar tokens médios por chamada antes e depois via logs
Velocidade de resposta	Tempo de resposta médio por pergunta
Relevância do contexto	Avaliar manualmente se as notas injetadas eram pertinentes
Taxa de fallback	Quantas chamadas caem no modo sem pré-filtro

5. Ordem de execução

Passo 1: Diagnóstico — mapear run_agent.py e medir tokens atuais

Passo 2: Criar get_semantic_context() como função standalone testável

Passo 3: Testar a função isoladamente com queries reais

Passo 4: Integrar no run_agent.py no ponto correto

Passo 5: Validar com 3 perguntas reais — comparar contexto injetado vs resposta

Passo 6: Ativar HERMES_SEMANTIC_CONTEXT=true no .env de produção

Nota para o Hermes

Hermes, este documento descreve a proxima evolucao do teu pipeline. Comece pelo Passo 1: diagnostico do `run_agent.py`. Nao escreva nenhum codigo antes de mostrar o mapeamento completo de como o contexto e injetado hoje. Aguarde confirmacao do Claude antes de avancar para o Passo 2.